Day 7 Task Workflow — Monday, Mar 3, 2026

> **Sprint:** 2 (Capture Engine + Core UI) | **Phase:** Capture Engine Foundation
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Scapy capture engine, flow aggregation, feature extraction pipeline, Redis pub/sub integration
> **Grade:** Week 1 A- COMPLETE ☑ | Week 2 STARTING ⬤

Day 7 Objective

Begin Week 2 by implementing the core capture engine foundation so that by end of day:

- Scapy capture engine operational on VPS network interfaces
- Flow aggregation logic grouping packets into bidirectional flows
- Feature extraction pipeline computing 40+ features per flow
- Redis pub/sub integration publishing flows to `flows:live` channel
- Capture engine control API endpoints (`/capture/*`)
- Database persistence of captured flows to `network_flows` table

Scope Adherence Checklist

> **STRICT RULE:** Every task below MUST adhere to the master documentation specifications. No features outside the defined scope. No Kafka, Kubernetes, Elasticsearch, or overengineered infrastructure.

Requirement	Source Document	Section
Scapy packet capture	MASTER_DOC_PART2	§2.1 (Tier 1: Capture Engine)
Flow aggregation (5-tuple)	MASTER_DOC_PART2	§2.1 (Flow Aggregator)
Feature extraction (40+)	MASTER_DOC_PART2	§2.1 (Feature Vector)
Redis pub/sub (flows:live)	MASTER_DOC_PART2	§6.1 (Redis Pub/Sub)
Capture API endpoints	MASTER_DOC_PART2	§5.1 (`/capture/*` endpoints)
PostgreSQL persistence	MASTER_DOC_PART2	§4.2 (network_flows table)
Docker privileged mode	MASTER_DOC_PART2	§3.2 (Docker Compose)
Host network mode	MASTER_DOC_PART2	§3.2 (Capture service)
Feature vector (JSONB)	MASTER_DOC_PART2	§4.2 (features JSONB)
Flow timeout (30s/120s)	MASTER_DOC_PART2	§2.1 (Flow Aggregator)

Architectural Constraints

> **ZERO TOLERANCE for deviation.** These are locked decisions from the master documentation.

Constraint	Rationale	Enforcement
Python 3.11+ only	Stack locked	requirements.txt
Scapy 2.5+ for capture	No tcpdump/windump external deps	Pure Python
Redis 7 pub/sub	Real-time pipeline, not Kafka	Docker Compose
PostgreSQL 16 for storage	JSONB for features, INET for IPs	SQLAlchemy 2.x
No external packet tools	Stack locked — Scapy only	Code review
Host network for capture	Raw socket access required	docker-compose.yml
Privileged container	Raw socket permissions	docker-compose.yml
5-tuple flow key	(src_ip, dst_ip, src_port, dst_port, protocol)	FlowAggregator
30s active timeout	Flow completion trigger	Config
120s idle timeout	Incomplete flow cleanup	Config

Task Breakdown

TASK 1 — Capture Engine Module Structure ⬛

Time Est: 30 min | **Priority:** ⬛ Critical

Create the capture engine module structure per MASTER_DOC_PART5 §2.1.

1.1 Create Capture Module (`backend/capture/`)

Files to create:

```
backend/capture/
├── __init__.py
├── engine.py               # Main capture loop (Scapy sniff)
├── flow_aggregator.py      # 5-tuple flow assembly
├── feature_extractor.py    # 40+ feature computation
└── config.py               # Capture configuration
```

Implementation Requirements:

`capture/__init__.py`:

```
"""ThreatMatrix AI — Capture Engine Module"""
```

`capture/config.py`:

```python
from pydantic import BaseSettings

class CaptureConfig(BaseSettings):
    interface: str = "eth0"
    active_timeout: float = 30.0      # Flow active timeout (seconds)
    idle_timeout: float = 120.0       # Flow idle timeout (seconds)
    bpf_filter: str = ""              # Berkeley Packet Filter
    max_flows_buffer: int = 10000     # Max flows in memory
    redis_channel: str = "flows:live" # Redis pub/sub channel
    batch_size: int = 50              # Flows per batch publish

    class Config:
        env_prefix = "CAPTURE_"
```

Verification:

Check	Command	Expected
Module structure	`ls backend/capture/`	4 files exist
Config imports	`python -c "from capture.config import CaptureConfig"`	No errors
Module imports	`python -c "import capture"`	No errors

TASK 2 — Flow Aggregator Implementation 🟧

Time Est: 60 min | **Priority:** 🟧 Critical

Implement the flow aggregation logic that groups packets into bidirectional flows.

2.1 Create Flow Aggregator (`backend/capture/flow_aggregator.py`)

Flow Key Structure (5-tuple):

```python
@dataclass
class FlowKey:
    src_ip: str
    dst_ip: str
    src_port: int
    dst_port: int
    protocol: int  # 6=TCP, 17=UDP, 1=ICMP

    def normalize(self) -> 'FlowKey':
        """Normalize to bidirectional: smaller IP first"""
        if self.src_ip < self.dst_ip:
            return self
        return FlowKey(
            src_ip=self.dst_ip,
            dst_ip=self.src_ip,
            src_port=self.dst_port,
```

```
            dst_port=self.src_port,
            protocol=self.protocol
        )
```

Flow Buffer Structure:

```
@dataclass
class FlowBuffer:
    key: FlowKey
    start_time: float
    last_seen: float
    src_packets: int = 0
    dst_packets: int = 0
    src_bytes: int = 0
    dst_bytes: int = 0
    packets: List[Packet] = field(default_factory=list)

    # TCP flag counts
    syn_count: int = 0
    ack_count: int = 0
    fin_count: int = 0
    rst_count: int = 0
    psh_count: int = 0
    urg_count: int = 0

    # Inter-arrival times
    inter_arrival_times: List[float] = field(default_factory=list)
    last_packet_time: Optional[float] = None

    # Payload data
    payload_bytes: List[bytes] = field(default_factory=list)
    payload_sizes: List[int] = field(default_factory=list)
```

Aggregator Class:

```
class FlowAggregator:
    def __init__(self, config: CaptureConfig):
        self.config = config
        self.flows: Dict[FlowKey, FlowBuffer] = {}
        self.completed_flows: List[FlowBuffer] = []

    def add_packet(self, packet: Packet) -> Optional[FlowBuffer]:
        """Add packet to flow buffer. Return completed flow if timeout reached."""
        flow_key = self._extract_flow_key(packet)
        if not flow_key:
            return None

        normalized_key = flow_key.normalize()
        current_time = time.time()
```

```
        if normalized_key not in self.flows:
            self.flows[normalized_key] = FlowBuffer(
                key=normalized_key,
                start_time=current_time,
                last_seen=current_time
            )

        flow = self.flows[normalized_key]

        # Update flow with packet data
        self._update_flow(flow, packet, current_time)

        # Check for flow completion
        if self._should_complete_flow(flow, current_time):
            del self.flows[normalized_key]
            return flow

        return None

    def flush_expired(self) -> List[FlowBuffer]:
        """Flush all flows that have exceeded idle timeout."""
        current_time = time.time()
        expired_keys = []

        for key, flow in self.flows.items():
            if current_time - flow.last_seen > self.config.idle_timeout:
                expired_keys.append(key)

        expired_flows = []
        for key in expired_keys:
            expired_flows.append(self.flows.pop(key))

        return expired_flows

    def _extract_flow_key(self, packet) -> Optional[FlowKey]:
        """Extract 5-tuple from Scapy packet."""
        # Implementation details...

    def _update_flow(self, flow: FlowBuffer, packet, timestamp: float):
        """Update flow buffer with packet data."""
        # Implementation details...

    def _should_complete_flow(self, flow: FlowBuffer, current_time: float) ->
bool:
        """Check if flow should be completed (FIN/RST or active timeout)."""
        # Implementation details...
```

Verification:

Check	Command	Expected

Check	Command	Expected
Module imports	`python -c "from capture.flow_aggregator import FlowAggregator"`	No errors
FlowKey creation	Unit test	5-tuple created correctly
Bidirectional norm	Unit test	Keys normalize correctly
Packet grouping	Unit test	Packets grouped by flow
Timeout detection	Unit test	Flows complete on timeout
FIN/RST detection	Unit test	Flows complete on FIN/RST

TASK 3 — Feature Extraction Pipeline 🟤

Time Est: 90 min | **Priority:** 🟤 Critical

Implement the feature extraction pipeline that computes 40+ features per completed flow.

3.1 Create Feature Extractor (`backend/capture/feature_extractor.py`)

Feature Categories (per MASTER_DOC_PART2 §2.1):

Category	Features	Count
Basic	duration, protocol_type, service, flag	4
Volume	src_bytes, dst_bytes, total_bytes, byte_ratio	4
Packet	src_packets, dst_packets, total_packets, packet_ratio	4
Timing	mean_iat, std_iat, min_iat, max_iat	4
TCP Flags	syn_count, ack_count, fin_count, rst_count, psh_count, urg_count	6
Connection	same_host_count, same_service_count, serror_rate, rerror_rate	4
DNS	query_count, response_count, unique_domains, avg_query_length	4
Payload	payload_entropy, mean_payload_size, has_payload	3
Behavioral	is_internal, port_class, geo_country	3
Statistical	flow_duration_zscore, byte_zscore, packet_zscore	3
Derived	packets_per_second, bytes_per_packet, connection_density	3+

Implementation:

```python
import numpy as np
from typing import Dict, Any
import math

class FeatureExtractor:
    """Extract ML-ready features from completed flows."""

    def extract(self, flow: FlowBuffer) -> Dict[str, Any]:
        """Extract all features from a completed flow."""
        features = {}

        # Basic features
        features['duration'] = flow.last_seen - flow.start_time
        features['protocol_type'] = self._get_protocol_name(flow.key.protocol)
        features['service'] = self._get_service(flow.key.dst_port)
        features['flag'] = self._get_tcp_flag_status(flow)

        # Volume features
        features['src_bytes'] = flow.src_bytes
        features['dst_bytes'] = flow.dst_bytes
        features['total_bytes'] = flow.src_bytes + flow.dst_bytes
        total = flow.src_bytes + flow.dst_bytes
        features['byte_ratio'] = flow.src_bytes / total if total > 0 else 0.5

        # Packet features
        features['src_packets'] = flow.src_packets
        features['dst_packets'] = flow.dst_packets
        features['total_packets'] = flow.src_packets + flow.dst_packets
        total_pkts = flow.src_packets + flow.dst_packets
        features['packet_ratio'] = flow.src_packets / total_pkts if total_pkts > 0
else 0.5

        # Timing features
        if flow.inter_arrival_times:
            iats = flow.inter_arrival_times
            features['mean_iat'] = float(np.mean(iats))
            features['std_iat'] = float(np.std(iats))
            features['min_iat'] = float(np.min(iats))
            features['max_iat'] = float(np.max(iats))
        else:
            features['mean_iat'] = 0.0
            features['std_iat'] = 0.0
            features['min_iat'] = 0.0
            features['max_iat'] = 0.0

        # TCP flag counts
        features['syn_count'] = flow.syn_count
        features['ack_count'] = flow.ack_count
        features['fin_count'] = flow.fin_count
        features['rst_count'] = flow.rst_count
        features['psh_count'] = flow.psh_count
        features['urg_count'] = flow.urg_count
```

```
        # Payload features
        features['payload_entropy'] = self._calculate_entropy(flow.payload_bytes)
        features['mean_payload_size'] = float(np.mean(flow.payload_sizes)) if
flow.payload_sizes else 0.0
        features['has_payload'] = len(flow.payload_bytes) > 0

        # Derived features
        duration = features['duration']
        features['packets_per_second'] = features['total_packets'] / max(duration,
0.001)
        features['bytes_per_packet'] = features['total_bytes'] /
max(features['total_packets'], 1)

        # Port classification
        features['port_class'] = self._classify_port(flow.key.dst_port)

        # Internal detection
        features['is_internal'] = self._is_internal_ip(flow.key.src_ip)

        return features

    def _calculate_entropy(self, data: bytes) -> float:
        """Calculate Shannon entropy of byte data."""
        if not data:
            return 0.0

        byte_counts = {}
        for byte in data:
            byte_counts[byte] = byte_counts.get(byte, 0) + 1

        entropy = 0.0
        data_len = len(data)
        for count in byte_counts.values():
            probability = count / data_len
            if probability > 0:
                entropy -= probability * math.log2(probability)

        return entropy

    def _get_service(self, port: int) -> str:
        """Map port to service name."""
        service_map = {
            80: 'http', 443: 'https', 22: 'ssh', 21: 'ftp',
            25: 'smtp', 53: 'dns', 3306: 'mysql', 5432: 'postgresql',
            6379: 'redis', 8000: 'http-alt', 8080: 'http-proxy'
        }
        return service_map.get(port, 'other')

    def _classify_port(self, port: int) -> str:
        """Classify port as well-known, registered, or dynamic."""
        if port < 1024:
            return 'well_known'
        elif port < 49152:
            return 'registered'
```

```
        else:
            return 'dynamic'

    def _is_internal_ip(self, ip: str) -> bool:
        """Check if IP is internal/private."""
        return (
            ip.startswith('10.') or
            ip.startswith('192.168.') or
            ip.startswith('172.16.') or
            ip.startswith('127.')
        )
```

Verification:

Check	Command	Expected
Module imports	`python -c "from capture.feature_extractor import FeatureExtractor"`	No errors
Feature count	Unit test	40+ features extracted
Entropy calculation	Unit test	Correct Shannon entropy
Service mapping	Unit test	Port 80 → 'http'
Port classification	Unit test	Port 80 → 'well_known'
Internal IP detection	Unit test	10.0.1.5 → True

TASK 4 — Redis Publisher Integration ⬛

Time Est: 30 min | **Priority:** ⬛ Critical

Integrate Redis pub/sub for publishing completed flows to the real-time pipeline.

4.1 Create Redis Publisher (`backend/capture/publisher.py`)

Architecture (per MASTER_DOC_PART2 §6.1):

```
Capture Engine ──► Redis Pub/Sub ──► FastAPI ──► WebSocket ──► Browser
                   (flows:live)      (subscriber)  (broadcast)  (N clients)
```

Implementation:

```python
import json
import redis.asyncio as redis
from typing import List, Dict, Any

class FlowPublisher:
    """Publish completed flows to Redis pub/sub and persist to PostgreSQL."""

    def __init__(self, redis_url: str, channel: str = "flows:live"):
        self.redis_url = redis_url
        self.channel = channel
        self.redis_client: Optional[redis.Redis] = None

    async def connect(self):
        """Establish Redis connection."""
        self.redis_client = redis.from_url(self.redis_url, decode_responses=True)

    async def publish_flow(self, flow_data: Dict[str, Any]):
        """Publish single flow to Redis channel."""
        if not self.redis_client:
            await self.connect()

        message = json.dumps({
            'event': 'new_flow',
            'payload': flow_data
        })

        await self.redis_client.publish(self.channel, message)

    async def publish_batch(self, flows: List[Dict[str, Any]]):
        """Publish batch of flows."""
        for flow in flows:
            await self.publish_flow(flow)

    async def publish_anomaly(self, flow_data: Dict[str, Any]):
        """Publish anomaly detection event."""
        if not self.redis_client:
            await self.connect()

        message = json.dumps({
            'event': 'anomaly_detected',
            'payload': flow_data
        })

        await self.redis_client.publish('ml:live', message)

    async def close(self):
        """Close Redis connection."""
        if self.redis_client:
            await self.redis_client.close()
```

Verification:

Check	Command	Expected
Module imports	`python -c "from capture.publisher import FlowPublisher"`	No errors
Redis connection	Unit test	Connected to Redis
Publish message	Integration test	Message in Redis channel
Message format	Unit test	JSON with event + payload

TASK 5 — Main Capture Engine ⬛

Time Est: 60 min | **Priority:** ⬛ Critical

Implement the main capture engine that orchestrates packet sniffing, flow aggregation, feature extraction, and publishing.

5.1 Create Capture Engine (`backend/capture/engine.py`)

Implementation:

```
import asyncio
import signal
import time
from typing import Optional
from scapy.all import sniff, IP, TCP, UDP, ICMP, Packet

from capture.config import CaptureConfig
from capture.flow_aggregator import FlowAggregator
from capture.feature_extractor import FeatureExtractor
from capture.publisher import FlowPublisher

class CaptureEngine:
    """Main capture engine orchestrating packet capture and flow processing."""

    def __init__(self, config: CaptureConfig):
        self.config = config
        self.aggregator = FlowAggregator(config)
        self.extractor = FeatureExtractor()
        self.publisher = FlowPublisher(
            redis_url=config.redis_url,
            channel=config.redis_channel
        )
        self.running = False
        self.stats = {
            'packets_captured': 0,
            'flows_completed': 0,
```

```
            'flows_published': 0,
            'start_time': None
        }

    async def start(self):
        """Start the capture engine."""
        self.running = True
        self.stats['start_time'] = time.time()

        # Connect to Redis
        await self.publisher.connect()

        # Start background tasks
        asyncio.create_task(self._flush_expired_flows())
        asyncio.create_task(self._report_stats())

        # Start packet capture (blocking — runs in executor)
        loop = asyncio.get_event_loop()
        await loop.run_in_executor(None, self._capture_loop)

    def _capture_loop(self):
        """Main packet capture loop using Scapy."""
        print(f"[Capture] Starting capture on interface: {self.config.interface}")
        print(f"[Capture] BPF filter: {self.config.bpf_filter or 'none'}")

        sniff(
            iface=self.config.interface,
            prn=self._process_packet,
            filter=self.config.bpf_filter or None,
            store=False,
            stop_filter=lambda _: not self.running
        )

    def _process_packet(self, packet: Packet):
        """Process a single captured packet."""
        self.stats['packets_captured'] += 1

        # Extract packet data for flow aggregation
        pkt_data = self._extract_packet_data(packet)
        if not pkt_data:
            return

        # Add to flow aggregator
        completed_flow = self.aggregator.add_packet(pkt_data)

        if completed_flow:
            self._handle_completed_flow(completed_flow)

    def _extract_packet_data(self, packet: Packet) -> Optional[dict]:
        """Extract relevant data from Scapy packet."""
        if not packet.haslayer(IP):
            return None

        ip_layer = packet[IP]
```

```
        pkt_data = {
            'src_ip': ip_layer.src,
            'dst_ip': ip_layer.dst,
            'protocol': ip_layer.proto,
            'timestamp': time.time(),
            'length': len(packet),
            'payload': bytes(packet.payload) if packet.payload else b''
        }

        # Extract port information for TCP/UDP
        if packet.haslayer(TCP):
            tcp = packet[TCP]
            pkt_data['src_port'] = tcp.sport
            pkt_data['dst_port'] = tcp.dport
            pkt_data['flags'] = tcp.flags
        elif packet.haslayer(UDP):
            udp = packet[UDP]
            pkt_data['src_port'] = udp.sport
            pkt_data['dst_port'] = udp.dport
            pkt_data['flags'] = 0
        elif packet.haslayer(ICMP):
            pkt_data['src_port'] = 0
            pkt_data['dst_port'] = 0
            pkt_data['flags'] = 0
        else:
            pkt_data['src_port'] = 0
            pkt_data['dst_port'] = 0
            pkt_data['flags'] = 0

        return pkt_data

    def _handle_completed_flow(self, flow):
        """Process completed flow: extract features, publish, persist."""
        # Extract features
        features = self.extractor.extract(flow)

        # Prepare flow record
        flow_record = {
            'timestamp': time.time(),
            'src_ip': flow.key.src_ip,
            'dst_ip': flow.key.dst_ip,
            'src_port': flow.key.src_port,
            'dst_port': flow.key.dst_port,
            'protocol': flow.key.protocol,
            'duration': features['duration'],
            'total_bytes': features['total_bytes'],
            'total_packets': features['total_packets'],
            'src_bytes': features['src_bytes'],
            'dst_bytes': features['dst_bytes'],
            'features': features,
            'source': 'live'
        }

        # Publish to Redis (async)
```

```
            asyncio.create_task(self._publish_flow(flow_record))

            self.stats['flows_completed'] += 1

        async def _publish_flow(self, flow_record: dict):
            """Publish flow to Redis and persist to database."""
            try:
                await self.publisher.publish_flow(flow_record)
                self.stats['flows_published'] += 1
            except Exception as e:
                print(f"[Capture] Publish error: {e}")

        async def _flush_expired_flows(self):
            """Periodically flush expired flows."""
            while self.running:
                await asyncio.sleep(5)
                expired = self.aggregator.flush_expired()
                for flow in expired:
                    self._handle_completed_flow(flow)

        async def _report_stats(self):
            """Periodically report capture statistics."""
            while self.running:
                await asyncio.sleep(30)
                elapsed = time.time() - self.stats['start_time']
                pps = self.stats['packets_captured'] / max(elapsed, 1)
                print(f"[Capture] Stats: {self.stats['packets_captured']} pkts | "
                      f"{self.stats['flows_completed']} flows | "
                      f"{self.stats['flows_published']} published | "
                      f"{pps:.1f} pps")

        def stop(self):
            """Stop the capture engine."""
            self.running = False
            print("[Capture] Stopping capture engine...")
```

Verification:

Check	Command	Expected
Module imports	`python -c "from capture.engine import CaptureEngine"`	No errors
Engine instantiation	Unit test	Engine created with config
Packet extraction	Unit test	IP/TCP/UDP data extracted
Flow completion	Integration test	Flow completes after timeout
Stats tracking	Unit test	Counters increment

TASK 6 — Capture API Endpoints 🔴

Time Est: 30 min | **Priority:** 🔴 Critical

Implement the capture control API endpoints per MASTER_DOC_PART2 §5.1.

6.1 Create Capture Routes (`backend/app/api/v1/capture.py`)

Endpoints (per MASTER_DOC_PART2 §5.1):

Method	Endpoint	Description
GET	`/capture/status`	Current capture session status
POST	`/capture/start`	Start packet capture
POST	`/capture/stop`	Stop packet capture
POST	`/capture/upload-pcap`	Upload PCAP for analysis
GET	`/capture/interfaces`	Available network interfaces

Implementation:

```
from fastapi import APIRouter, Depends, HTTPException, UploadFile, File
from app.dependencies import get_current_user
from app.models.user import User
from app.schemas.capture import CaptureStatus, CaptureStartRequest,
CaptureStartResponse

router = APIRouter(prefix="/capture", tags=["Capture"])

# Global capture engine instance
_capture_engine: Optional[CaptureEngine] = None

@router.get("/status", response_model=CaptureStatus)
async def get_capture_status(
    user: User = Depends(get_current_user)
):
    """Get current capture session status."""
    global _capture_engine

    if not _capture_engine or not _capture_engine.running:
        return CaptureStatus(
            status="stopped",
            packets_captured=0,
            flows_completed=0,
            uptime=0
        )

    return CaptureStatus(
        status="running",
```

```
        interface=_capture_engine.config.interface,
        packets_captured=_capture_engine.stats['packets_captured'],
        flows_completed=_capture_engine.stats['flows_completed'],
        flows_published=_capture_engine.stats['flows_published'],
        uptime=time.time() - _capture_engine.stats['start_time']
    )

@router.post("/start", response_model=CaptureStartResponse)
async def start_capture(
    request: CaptureStartRequest,
    user: User = Depends(get_current_user)
):
    """Start packet capture on specified interface."""
    global _capture_engine

    # RBAC: Only admin and soc_manager can start capture
    if user.role not in ['admin', 'soc_manager']:
        raise HTTPException(status_code=403, detail="Insufficient permissions")

    if _capture_engine and _capture_engine.running:
        raise HTTPException(status_code=400, detail="Capture already running")

    config = CaptureConfig(
        interface=request.interface or "eth0",
        bpf_filter=request.bpf_filter or ""
    )

    _capture_engine = CaptureEngine(config)
    asyncio.create_task(_capture_engine.start())

    return CaptureStartResponse(
        status="started",
        interface=config.interface,
        message=f"Capture started on {config.interface}"
    )

@router.post("/stop")
async def stop_capture(
    user: User = Depends(get_current_user)
):
    """Stop packet capture."""
    global _capture_engine

    # RBAC: Only admin and soc_manager can stop capture
    if user.role not in ['admin', 'soc_manager']:
        raise HTTPException(status_code=403, detail="Insufficient permissions")

    if not _capture_engine or not _capture_engine.running:
        raise HTTPException(status_code=400, detail="Capture not running")

    _capture_engine.stop()

    return {"status": "stopped", "message": "Capture engine stopped"}
```

```python
@router.get("/interfaces")
async def list_interfaces(
    user: Depends(get_current_user)
):
    """List available network interfaces."""
    from scapy.all import get_if_list

    interfaces = []
    for iface in get_if_list():
        interfaces.append({
            "name": iface,
            "description": f"Network interface: {iface}"
        })

    return {"interfaces": interfaces}

@router.post("/upload-pcap")
async def upload_pcap(
    file: UploadFile = File(...),
    user: User = Depends(get_current_user)
):
    """Upload PCAP file for analysis."""
    # RBAC: admin, soc_manager, analyst can upload
    if user.role not in ['admin', 'soc_manager', 'analyst']:
        raise HTTPException(status_code=403, detail="Insufficient permissions")

    # Save file and process asynchronously
    file_path = f"pcaps/{file.filename}"
    # ... implementation

    return {"status": "uploaded", "filename": file.filename}
```

Schemas (`backend/app/schemas/capture.py`):

```python
from pydantic import BaseModel
from typing import Optional

class CaptureStatus(BaseModel):
    status: str
    interface: Optional[str] = None
    packets_captured: int = 0
    flows_completed: int = 0
    flows_published: int = 0
    uptime: float = 0

class CaptureStartRequest(BaseModel):
    interface: Optional[str] = "eth0"
    bpf_filter: Optional[str] = ""

class CaptureStartResponse(BaseModel):
    status: str
```

```
    interface: str
    message: str
```

Verification:

Check	Command	Expected
Endpoints mounted	Check `/docs`	5 capture endpoints
Status endpoint	`curl /api/v1/capture/status`	Status object
Start endpoint	`POST /api/v1/capture/start`	Started response
RBAC enforced	Viewer cannot start	403 Forbidden
Interfaces list	`curl /api/v1/capture/interfaces`	Interface list

TASK 7 — Database Persistence ⬛

Time Est: 30 min | **Priority:** ⬛ Critical

Implement database persistence for captured flows to the `network_flows` table.

7.1 Create Flow Persistence Service (`backend/app/services/flow_persistence.py`)

Database Table (per MASTER_DOC_PART2 §4.2):

```sql
CREATE TABLE network_flows (
    id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    timestamp       TIMESTAMPTZ NOT NULL,
    src_ip          INET NOT NULL,
    dst_ip          INET NOT NULL,
    src_port        INTEGER,
    dst_port        INTEGER,
    protocol        SMALLINT NOT NULL,
    duration        REAL,
    total_bytes     BIGINT,
    total_packets   INTEGER,
    src_bytes       BIGINT,
    dst_bytes       BIGINT,
    features        JSONB NOT NULL,
    anomaly_score   REAL,
    is_anomaly      BOOLEAN DEFAULT false,
    ml_model        VARCHAR(50),
    label           VARCHAR(50),
    source          VARCHAR(20) DEFAULT 'live',
    created_at      TIMESTAMPTZ DEFAULT NOW()
);
```

Implementation:

```python
from sqlalchemy.ext.asyncio import AsyncSession
from app.models.flow import NetworkFlow
from datetime import datetime
from typing import Dict, Any

class FlowPersistence:
    """Persist captured flows to PostgreSQL."""

    def __init__(self, session: AsyncSession):
        self.session = session

    async def save_flow(self, flow_data: Dict[str, Any]) -> NetworkFlow:
        """Save a single flow record."""
        flow = NetworkFlow(
            timestamp=datetime.fromtimestamp(flow_data['timestamp']),
            src_ip=flow_data['src_ip'],
            dst_ip=flow_data['dst_ip'],
            src_port=flow_data['src_port'],
            dst_port=flow_data['dst_port'],
            protocol=flow_data['protocol'],
            duration=flow_data['duration'],
            total_bytes=flow_data['total_bytes'],
            total_packets=flow_data['total_packets'],
            src_bytes=flow_data['src_bytes'],
            dst_bytes=flow_data['dst_bytes'],
            features=flow_data['features'],
            source=flow_data.get('source', 'live')
        )

        self.session.add(flow)
        await self.session.commit()
        await self.session.refresh(flow)

        return flow

    async def save_batch(self, flows: list[Dict[str, Any]]) -> int:
        """Save batch of flows. Returns count saved."""
        count = 0
        for flow_data in flows:
            await self.save_flow(flow_data)
            count += 1
        return count
```

Integration with Capture Engine:

```python
# In engine.py, update _publish_flow method:
async def _publish_flow(self, flow_record: dict):
    """Publish flow to Redis and persist to database."""
    try:
        # Publish to Redis for real-time
```

```
        await self.publisher.publish_flow(flow_record)

        # Persist to PostgreSQL
        async with get_db_session() as session:
            persistence = FlowPersistence(session)
            await persistence.save_flow(flow_record)

        self.stats['flows_published'] += 1
    except Exception as e:
        print(f"[Capture] Publish error: {e}")
```

Verification:

Check	Command	Expected
Service imports	`python -c "from app.services.flow_persistence import FlowPersistence"`	No errors
Flow model exists	Check `app/models/flow.py`	NetworkFlow model defined
Save flow	Integration test	Flow persisted to DB
JSONB features	Check DB	Features stored as JSONB
INET types	Check DB	IPs stored as INET

TASK 8 — Docker Compose Update 🟢

Time Est: 15 min | **Priority:** 🟢 Medium

Update Docker Compose to include the capture engine service.

8.1 Update `docker-compose.yml`

Add capture service (per MASTER_DOC_PART2 §3.2):

```
capture:
  build: ./backend
  network_mode: host
  privileged: true
  environment:
    REDIS_URL: redis://localhost:6379
    DATABASE_URL:
postgresql://threatmatrix:${DB_PASSWORD}@localhost:5432/threatmatrix
    CAPTURE_INTERFACE: ${CAPTURE_INTERFACE:-eth0}
    CAPTURE_ACTIVE_TIMEOUT: "30"
    CAPTURE_IDLE_TIMEOUT: "120"
  volumes:
    - ./pcaps:/app/pcaps
```

```
    command: python -m capture.engine
    depends_on:
      - redis
      - postgres
    restart: unless-stopped
```

Verification:

Check	Command	Expected
YAML valid	`docker-compose config`	No errors
Service defined	`docker-compose config`	capture service present
Host network	Check config	network_mode: host
Privileged	Check config	privileged: true

Files Created/Modified Today

```
threatmatrix-ai/
├── backend/
│   ├── capture/
│   │   ├── __init__.py                 🔨 TASK 1
│   │   ├── config.py                   🔨 TASK 1
│   │   ├── flow_aggregator.py          🔨 TASK 2
│   │   ├── feature_extractor.py        🔨 TASK 3
│   │   ├── publisher.py                🔨 TASK 4
│   │   └── engine.py                   🔨 TASK 5
│   ├── app/
│   │   ├── api/v1/
│   │   │   └── capture.py              🔨 TASK 6
│   │   ├── schemas/
│   │   │   └── capture.py              🔨 TASK 6
│   │   └── services/
│   │       └── flow_persistence.py     🔨 TASK 7
│   └── requirements.txt                🔨 Update (add scapy)
├── docker-compose.yml                  🔨 TASK 8 (update)
└── docs/
    └── worklog/
        └── DAY_07_MAR03.md             🔨 This file
```

Verification Checklist

> **Every item below MUST be verified before marking task complete.**

#	Verification	Command	Expected Result

#	Verification	Command	Expected Result
1	Capture module exists	`ls backend/capture/`	6 files
2	Config imports	`python -c "from capture.config import CaptureConfig"`	No errors
3	Aggregator imports	`python -c "from capture.flow_aggregator import FlowAggregator"`	No errors
4	Extractor imports	`python -c "from capture.feature_extractor import FeatureExtractor"`	No errors
5	Publisher imports	`python -c "from capture.publisher import FlowPublisher"`	No errors
6	Engine imports	`python -c "from capture.engine import CaptureEngine"`	No errors
7	Capture API mounted	Check `/docs`	5 capture endpoints
8	Status endpoint	`curl /api/v1/capture/status`	Status object
9	Interfaces endpoint	`curl /api/v1/capture/interfaces`	Interface list
10	RBAC enforced	Viewer cannot start capture	403 Forbidden
11	Flow key normalization	Unit test	Bidirectional key
12	Feature count	Unit test	40+ features
13	Entropy calculation	Unit test	Correct Shannon entropy
14	Redis publish	Integration test	Message in channel
15	Flow persistence	Integration test	Flow in database
16	Docker config valid	`docker-compose config`	No errors
17	Scapy in requirements	`grep scapy requirements.txt`	scapy>=2.5
18	Python type hints	Code review	All functions typed
19	Async/await used	Code review	All I/O async

#	Verification	Command	Expected Result
20	Error handling	Code review	Try/except blocks

Scope Adherence Verification

Requirement	Source	Verification
Scapy capture	MASTER_DOC_PART2 §2.1	engine.py uses scapy.sniff()
5-tuple flow key	MASTER_DOC_PART2 §2.1	FlowKey with src/dst ip/port/proto
40+ features	MASTER_DOC_PART2 §2.1	feature_extractor.py extracts 40+
Redis pub/sub	MASTER_DOC_PART2 §6.1	publisher.py publishes to flows:live
Capture endpoints	MASTER_DOC_PART2 §5.1	5 endpoints match spec
PostgreSQL JSONB	MASTER_DOC_PART2 §4.2	features stored as JSONB
INET types	MASTER_DOC_PART2 §4.2	IPs stored as INET
Host network	MASTER_DOC_PART2 §3.2	docker-compose: network_mode: host
Privileged mode	MASTER_DOC_PART2 §3.2	docker-compose: privileged: true
Flow timeout 30s	MASTER_DOC_PART2 §2.1	active_timeout = 30.0
Idle timeout 120s	MASTER_DOC_PART2 §2.1	idle_timeout = 120.0
Pure Python	STACK LOCKED	No tcpdump/windump
Python 3.11+	STACK LOCKED	Type hints used
SQLAlchemy 2.x	STACK LOCKED	mapped_column used

Blockers

Blocker	Severity	Mitigation	Status
No root access on Windows	Medium	Use `--privileged` in Docker	Expected
Scapy on Windows limitations	Medium	Capture runs in Docker (Linux)	Expected
No raw socket on Windows	Medium	Docker host networking solves this	Expected
Next.js 16 build error	Medium	Use `npm run dev` for development	Known bug

Tomorrow's Preview (Day 8 — Week 2 Day 2)

Per MASTER_DOC_PART5 §3 Week 2 plan:

- Capture engine refinement and testing
- Feature extraction validation against NSL-KDD format

- War Room: ThreatMap (Deck.gl + Maplibre) — Full-Stack Dev
- War Room: MetricCard connections — Full-Stack Dev
- Network Flow module: basic layout — Full-Stack Dev

Reference Documents

Document	Section	Purpose
MASTER_DOC_PART2	§2.1	Capture engine architecture
MASTER_DOC_PART2	§3.2	Docker Compose service config
MASTER_DOC_PART2	§4.2	Database schema (network_flows)
MASTER_DOC_PART2	§5.1	Capture API endpoints
MASTER_DOC_PART2	§6.1	Redis pub/sub architecture
MASTER_DOC_PART4	§3.1	Feature engineering pipeline
MASTER_DOC_PART4	§3.2	Feature extraction from Scapy
MASTER_DOC_PART4	§8.1	Real-time inference pipeline
MASTER_DOC_PART5	§2.1	Project structure (capture/)
MASTER_DOC_PART5	§3	Week 2 plan

Task workflow for Day 7 (Week 2 Day 1) — ThreatMatrix AI Sprint 2
Focus: Capture Engine Foundation